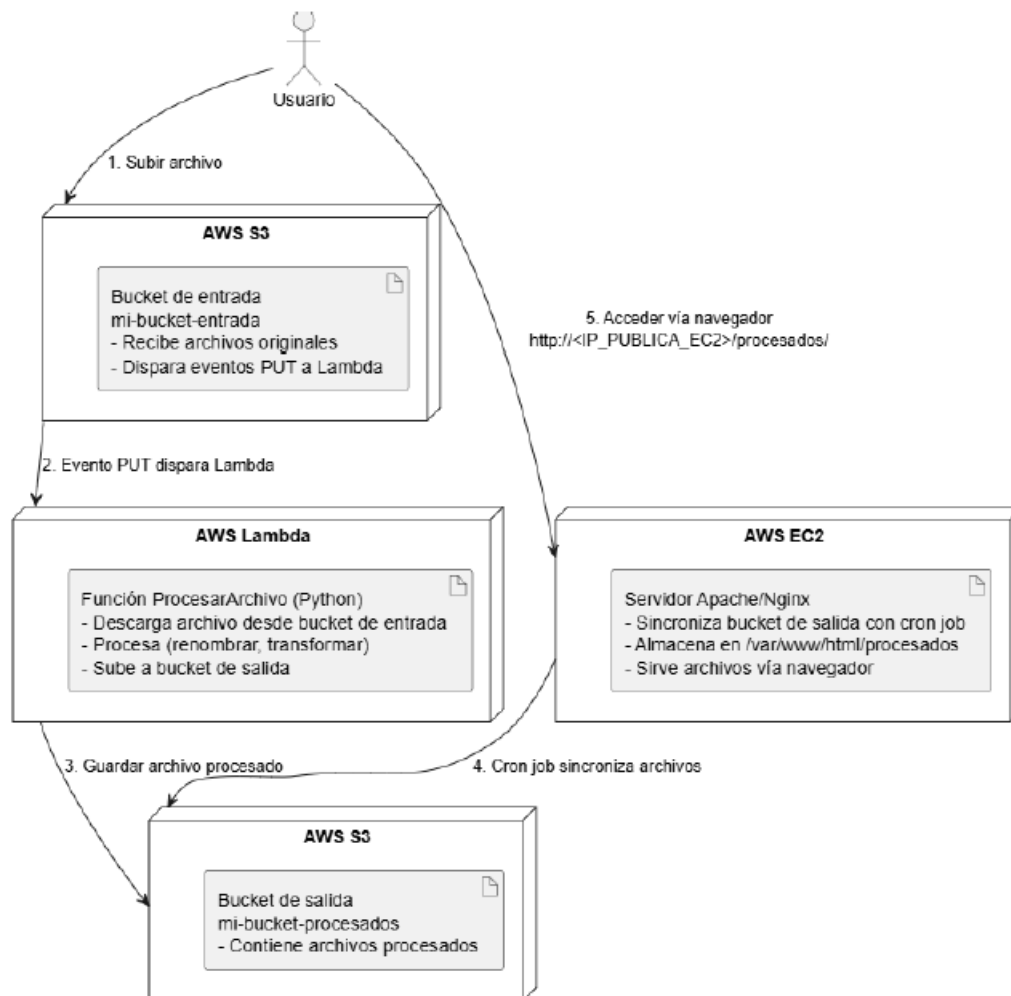


Práctica de AWS Academy: Automatización de archivos con S3, Lambda y EC2 (Ubuntu)



Objetivo

Construir una arquitectura en AWS que permita:

- Subir archivos a un **bucket S3**.
- Ejecutar automáticamente una **Lambda** que procese los archivos y los mueva a una carpeta procesado/.
- Configurar una **instancia EC2 Ubuntu con Apache** para que los archivos procesados estén disponibles para descarga desde un navegador.
- Usar los permisos proporcionados por **LabRole de AWS Academy**.

Servicios a utilizar

Amazon S3 → almacenamiento de archivos.

AWS Lambda → procesamiento automático de archivos.

IAM / LabRole → permisos para Lambda y EC2.

Amazon EC2 (Ubuntu) → servidor web con Apache.

CloudWatch Logs → visualizar la ejecución de Lambda.

Parte 1: Crear el bucket S3

Entra en **Amazon S3** → **Crear bucket**

Nombre: asir-practica-<tu-nombre>

Región: la misma donde crearás Lambda y EC2.

Crea dos carpetas dentro del bucket:

uploads/ → subirán los archivos originales.

procesado/ → Lambda guardará los archivos procesados.

Sube un archivo de prueba a uploads/ (por ejemplo datos.txt).

Parte 2: Crear la función Lambda

En **AWS Lambda** → **Crear función**

Nombre: procesar-archivos-s3

Runtime: Python 3.12

Rol: **LabRole** (ya provisto por AWS Academy).

Pega el siguiente código:

```
import boto3

s3 = boto3.client('s3')

def lambda_handler(event, context):
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']
    print(f"Archivo recibido: {key} en el bucket: {bucket}")

    # Leer el archivo (opcional, si quieres procesarlo)
    # response = s3.get_object(Bucket=bucket, Key=key)
    # data = response['Body'].read().decode('utf-8')
    # print("Contenido del archivo:")
    # print(data)

    # Crear nuevo nombre y ruta
    nuevo_nombre = f"procesado/{key.split('/')[-1]}"
    print(f"Nuevo archivo se guardará como: {nuevo_nombre}")
```

```
# Copiar el archivo a la carpeta "procesado/" con el nuevo nombre
s3.copy_object(
    Bucket=bucket,
    CopySource={'Bucket': bucket, 'Key': key},
    Key=nuevo_nombre
)
# (Opcional) Borrar el archivo original
# s3.delete_object(Bucket=bucket, Key=key)
# print(f"Archivo original eliminado: {key}")

return {
    'statusCode': 200,
    'body': f"Archivo {key} procesado y guardado como {nuevo_nombre}"
}
```

Permisos de Lambda (LabRole)

El **LabRole** ya tiene permisos básicos.

Parte 3: Configurar trigger S3 → Lambda

En el bucket, ve a **Propiedades → Eventos → Crear evento**.

Configura:

Nombre: trigger-lambda

Evento: "Put"

Destino: función Lambda procesar-archivos-s3

Guarda y prueba subiendo un archivo a uploads/.

Se generará automáticamente en procesado/ como procesado_<archivo>.

Parte 4: Crear EC2 Ubuntu con Apache

En **EC2 → Launch instance**

AMI: **Ubuntu 22.04 LTS**

Tipo: t3.micro

Par de claves: el que tengas en LabRole

Grupo de seguridad: abrir **puerto 80 (HTTP)**

Lanza la instancia y conéctate por SSH:

```
ssh -i "tu_clave.pem" ubuntu@<IP_Pública_EC2>
```

Parte 5: Instalar y configurar Apache

```
sudo apt update -y
sudo apt install -y apache2 unzip curl
sudo systemctl enable apache2
sudo systemctl start apache2
```

Parte 6: Script de sincronización S3 → Apache

Crea el script `setup_s3_apache.sh` en tu EC2:

```
GNU nano 7.2 script.sh *
#!/bin/bash

# =====
# Script para servir archivos S3 en Apache
# =====

# Variables
BUCKET="s3-590183782760"
CARPETA="/var/www/html/procesado"

# 1 Crear la carpeta de destino si no existe
sudo mkdir -p $CARPETA
```

```
# 2 Sincronizar archivos desde S3
aws s3 sync s3://$BUCKET/procesado/ $CARPETA --delete
```

```
# 3 Ajustar permisos para Apache
sudo chown -R www-data:www-data $CARPETA
sudo chmod -R 755 $CARPETA
```

```
# 4 Reiniciar Apache para aplicar cambios
sudo systemctl restart apache2
```

Ejecutar script

```
chmod +x script.sh
./script.sh
```

--delete

Este flag indica que si hay archivos **en la carpeta local** que ya no existen en S3, se **eliminarán automáticamente**.

Mantiene la carpeta local exactamente igual que el bucket S3.

Evita que queden archivos antiguos que ya no se usan.

Cron para sincronización automática. Editar el crontab

Ejecuta:

```
crontab -e
```

Esto abre el editor del cron de tu usuario (ubuntu).

3 Agregar la línea para sincronización automática

Al final del archivo que se abre, agrega:

```
* * * * * /home/ubuntu/script.sh
```

Esto significa: **cada minuto**, ejecuta tu script. Asegúrate de que la ruta sea correcta y que el script tenga permisos de ejecución . Esto sincroniza los archivos procesados cada minuto.

Si quieres sincronizar manualmente

```
./script.sh
```

Objetivos de aprendizaje

Entender el flujo **S3** → **Lambda** → **S3**.

Configurar triggers automáticos y visualizar logs en **CloudWatch**.

Configurar EC2 Ubuntu con **Apache** para servir archivos de S3.

Aplicar permisos de **LabRole** para Lambda y EC2.

Prueba final

Subir archivo a uploads/ → Lambda procesa y genera procesado_<archivo> en procesado/.

En el navegador:

`http://<IP_Pública_EC2>/procesado/`
Deberías ver y poder descargar el archivo procesado.



 No es seguro 54.227.8.201/procesado/

Index of /procesado

	<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
	Parent Directory		-	
	procesado	2025-10-13 18:50	0	
	procesado_Hola.txt	2025-10-13 17:41	0	
	procesado_Hola2.txt	2025-10-13 18:38	0	
	procesado_pasos.txt	2025-10-13 17:21	1.1K	

Apache/2.4.58 (Ubuntu) Server at 54.227.8.201 Port 80